

# 허밍 기반 릴스.쇼츠 밈 구간 검색 프로젝트

## 결과보고서

### 프로젝트 개요

본 프로젝트는 사용자가 업로드한 허밍 .WAV 파일만으로, 숏폼 플랫폼에서 자주 쓰이는 배속.피치 변형.밈 음원의 원곡 정보와 음원과 유사한 영상을 찾고 사용 맥락을 함께 분석하는 시스템이다. 단순히 곡 제목을 찾는 데서 멈추지 않고, 원곡의 어느 구간이 가까운지, 원곡 대비 얼마나 빨라졌는지, 몇 반음 정도 올라가거나 내려갔는지, 멜로디.화성 구조가 얼마나 유지되는지까지 계산하고, 이후 YouTube 후보와 트렌드 리포트까지 연결한다. 전체 사용자 흐름은 Streamlit UI 에서 원곡 정보, 변형 분석, YouTube 후보, 트렌드 리포트, 전체 세그먼트 유사도 순으로 제시되도록 구현되어 있다.

이번 실험 구성에서 원곡 DB 는 Justin Bieber - Beauty And A Beat (feat. Nicki Minaj)와 Hearts2Hearts - RUDE! 두 곡으로 구성되었고, 테스트 입력은 Beauty and a Beat 의 릴스형 허밍 데이터였다. 코드상 실제 곡 메타데이터는 수동 카탈로그로 관리되며, 현재 등록된 곡도 위 두 곡이다.

아래 표는 이 프로젝트를 결과 중심으로 요약한 것이다.

| 항목        | 내용                                                                  |
|-----------|---------------------------------------------------------------------|
| 분석 대상     | 허밍/짧은 녹음 WAV                                                        |
| DB 저장 방식  | 원곡 WAV 만 저장                                                         |
| 검색 단위     | 원곡 10 초 세그먼트                                                        |
| 특징 벡터     | MFCC + Chroma + Pitch contour                                       |
| 정밀 판별     | speed factor, pitch shift, chroma similarity, transform match score |
| 부가 출력     | YouTube 후보 영상, GPT 기반 트렌드 리포트, fallback 리포트                         |
| 사용자 인터페이스 | 원곡 정보 → 변형 음원 → 변형 분석 → 실제 사용 영상 → 트렌드 리포트                          |

표의 항목은 실제 파이프라인 구현과 결과 화면 구성 기준으로 정리했다.

## 핵심 코드

```
from app.search import search_reel_segment, build_cache
from app.gpt import describe_song, build_fallback_report
from app.youtube import search_youtube_multi, rank_youtube_matches
```

이 세 줄은 이 프로젝트의 핵심 연결 구조를 가장 압축적으로 보여준다. 검색, 트렌드 리포트, 외부 후보 수집이 하나의 UI 안에서 통합된다.

## 문제 정의와 차별점

본 프로젝트의 핵심 문제의식은 숏폼 콘텐츠에서 사용되는 변형 음원을 단순한 곡명 검색이 아니라, 원곡 대비 변형 맥락까지 설명 가능한 형태로 분석하는 데 있다. 릴스나 쇼츠의 음원은 원곡 그대로 사용되기보다 배속, 피치 변환, 짧은 하이라이트 반복, 밌형 편집 등의 형태로 재가공되는 경우가 많다. 이러한 환경에서는 원곡 제목만 반환하는 기존 검색 방식으로는 사용자가 실제로 들은 버전의 특성과 숏폼 활용 맥락을 충분히 설명하기 어렵다. 이에 본 프로젝트는 원곡 세그먼트만 저장한 뒤 입력 오디오와의 유사도 및 차이를 계산하여 speed factor, pitch shift, chroma similarity 등을 도출하고, 이를 바탕으로 변형 음원의 성격과 밌적 활용 가능성을 설명하는 시스템을 구현하였다.

가장 큰 차별점은 리믹스 파일을 DB 에 전부 저장하지 않는다는 점이다.

test\_segment.py 는 \_original.wav 로 끝나는 원곡만 세그먼트화하고, 검색 단계 역시 \_is\_catalog\_original\_segment()를 통과한 원곡 세그먼트만 대상으로 비교한다. 다시 말해, DB 는 원곡 중심이고, 변형 여부는 검색 이후의 정밀 재매칭에서 판별한다. 이 구조는 리믹스 종류가 늘어날수록 DB 규모가 비대해지는 문제를 줄여 준다.

또 다른 차별점은 “오디오만 사용한다”는 점이다. 업로드 파일은 임시 WAV 로 저장된 뒤 바로 search\_reel\_segment()에 들어가며, 검색 함수는 파일명 의미를 해석하지 않고 오디오 특징 벡터만 계산한다. 즉, 제목 힌트 없이도 원곡-변형 관계를 찾도록 설계되어 있다.

특히 의미 있는 차별점은 “1 차 원형 매칭”과 “2 차 변형 정밀 매칭”을 분리했다는 점이다. 이 구조 덕분에 기본 코사인 유사도만 보면 낮게 보였던 후보라도, speed/pitch 보정 이후 더 높은 최종 점수로 뒤집을 수 있다. 실행 화면에서도 1 차 원형 매칭 점수 73.6%가 변형

후보 정밀 매칭 88.5%로 상승해, 단순 원형 매칭만으로는 놓칠 수 있는 샷폼 변형 음원을 보완하고 있음을 보여 주었다. 이 부분은 본 프로젝트의 가장 강한 차별점이다.

### 핵심 코드

```
INPUT_FILES = [  
    os.path.join(SONGS_FOLDER, filename)  
    for filename in sorted(os.listdir(SONGS_FOLDER))  
    if filename.lower().endswith("_original.wav")  
]  
  
wav_files = sorted([  
    f for f in os.listdir(segments_folder)  
    if f.lower().endswith(".wav") and _is_catalog_original_segment(f)  
])  
  
for speed in SPEED_CANDIDATES:  
    for pitch_shift in PITCH_CANDIDATES:  
        transformed = _apply_transform(y, sr, speed, pitch_shift)  
        transform_vec = extract_features_from_audio(transformed, sr)
```

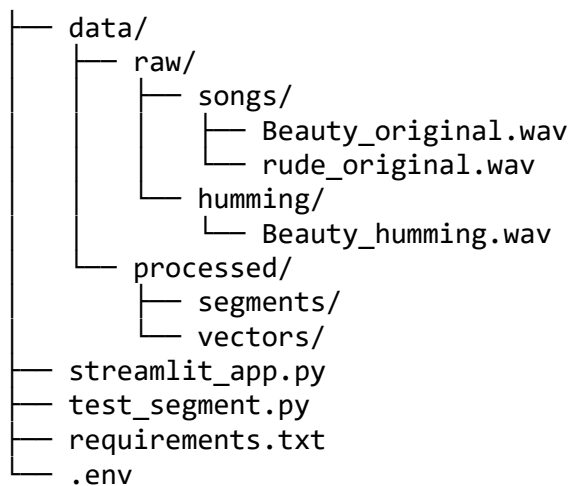
첫 번째 코드 조각은 원곡만 세그먼트화하는 규칙을, 두 번째는 검색도 원곡 세그먼트에 한정된다는 점을, 세 번째는 변형 후보를 “저장”하는 대신 “실시간 생성”해 다시 비교한다는 차별점을 보여 준다.

## 데이터 구성과 폴더 구조

이번 프로젝트의 데이터는 원곡 데이터와 허밍 데이터로 나뉜다. 원곡 데이터는 raw/songs 에 저장되고, 테스트 입력은 허밍 또는 짧은 녹음 WAV 로 들어온다. 검색 전에 원곡 WAV 는 세그먼트 파일로 분할되고, 세그먼트별 특징 벡터는 별도 캐시에 저장된다. 또한 실제 곡명과 아티스트 정보는 별도 카탈로그 파일에서 관리되어, 로컬 파일명과 화면 표시용 메타데이터를 분리했다.

보고서에 제시하기 좋은 프로젝트 폴더 구조는 아래와 같다.

```
music_ai_project/  
├── app/  
│   ├── audio.py  
│   ├── segment.py  
│   ├── search.py  
│   ├── song_catalog.py  
│   ├── youtube.py  
│   └── gpt.py
```



이 구조는 모듈 import 경로, 원곡·세그먼트·캐시 경로, 실행 스크립트, 환경변수 파일을 종합해 정리한 것이다.

원곡 파일과 실제 곡 정보의 연결은 `song_catalog.py` 가 담당한다. 현재 실험용 DB 는 `beauty` 와 `rude` 두 항목으로 등록되어 있으며, 이 매핑 덕분에 화면에는 로컬 파일명이 아니라 실제 곡 제목과 아티스트가 표시된다.

## 핵심 코드

```
SONG_CATALOG = {
    "beauty": {
        "original_file_stem": "Beauty_original",
        "artist": "Justin Bieber",
        "featured_artist": "Nicki Minaj",
        "full_title": "Beauty And A Beat (feat. Nicki Minaj)",
        "youtube_query": "Justin Bieber Beauty And A Beat ft Nicki Minaj",
    },
    "rude": {
        "original_file_stem": "rude_original",
        "artist": "Hearts2Hearts",
        "full_title": "RUDE!",
        "youtube_query": "Hearts2Hearts RUDE!",
    }
}

SEGMENTS_FOLDER = "data/processed/segments"
SONGS_FOLDER     = "data/raw/songs"
VECTOR_CACHE_DIR = "data/processed/vectors"
```

위 코드는 실험 데이터와 폴더 구조가 어떻게 동작하는지를 가장 직접적으로 보여 준다.

또한 외부 연동은 환경변수 기반으로 분리되어 있다. OpenAI 게이트웨이와 YouTube Data API 키를 코드에 직접 쓰지 않고 .env 에서 읽어 오도록 구현했고, 실행 스택은 librosa, numpy, scikit-learn, streamlit, openai, httpx 등을 사용한다.

## 핵심 구현과 알고리즘

이 프로젝트의 핵심 알고리즘은 허밍 입력과 원곡의 특정 구간을 비교하여 가장 유사한 음악 구간과 변형 유형을 찾는 방식으로 구성된다. 전체 과정은 크게 특징 추출, 세그먼트 기반 검색, 정밀 매칭, 결과 리포트 생성의 네 단계로 이루어진다.

먼저 원곡 오디오는 10 초 단위 세그먼트로 분할되고, 각 세그먼트와 사용자 허밍에서 특징 벡터를 추출한다. 특징 벡터는 MFCC, chroma, pitch contour 를 결합하여 생성한다. MFCC 는 음색과 배음 구조를, chroma 는 음계 분포를, pitch contour 는 음높이 변화 패턴을 반영한다. 각 특징은 정규화 후 결합되며, chroma 와 pitch 에는 추가 가중치를 부여하여 멜로디 정보가 더 잘 반영되도록 설계하였다.

### 핵심 코드

```
mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=n_mfcc)
mfcc_vec = _l2(np.mean(mfcc, axis=1))
```

```
chroma = librosa.feature.chroma_stft(y=y, sr=sr)
chroma_vec = _l2(np.mean(chroma, axis=1)) * 2.0
```

```
pitch_vec = _extract_pitch_contour(y, sr, n_bins=20) * 2.5
```

```
combined = np.concatenate([mfcc_vec, chroma_vec, pitch_vec])
return _l2(combined)
```

이 과정을 통해 최종적으로 45 차원의 특징 벡터가 생성되며, 단순 음색뿐 아니라 멜로디와 음높이 변화까지 함께 반영할 수 있다. 이는 배속·감속·피치 변형이 자주 발생하는 샷폼 음원 검색에 유리하게 작용한다.

세그먼트 생성 단계에서는 원곡을 일정 길이로 분할하여 검색 단위를 만든다. 현재 구현은 10 초 길이의 비중첩 세그먼트를 사용하며, 각 세그먼트에 대해 특징 벡터를 미리 계산하여 저장한다.

### 핵심 코드

```
SEGMENT_DURATION = 10
OVERLAP_DURATION = 0
STEP_DURATION = SEGMENT_DURATION - OVERLAP_DURATION
while start_sample + segment_samples <= len(y):
    end_sample = start_sample + segment_samples
    segment = y[start_sample:end_sample]
```

검색 단계에서는 입력 허밍과 모든 세그먼트 간 코사인 유사도를 계산하여 상위 후보를 찾는다. 이후 상위 후보에 대해서만 추가 분석을 수행하는 방식으로 연산량을 줄이면서도 정확도를 확보한다.

정밀 매칭 단계에서는 배속 및 피치 변형을 고려한다. 상위 후보 다섯 개에 대해 다양한 speed factor 와 pitch shift 를 적용하여 가장 높은 유사도를 찾고, BPM 및 chroma 분석을 통해 실제 스포츠에서 사용된 변형 형태를 추정한다.

### 핵심 코드

```
SPEED_CANDIDATES = [0.85, 1.0, 1.12, 1.25, 1.35]
PITCH_CANDIDATES = [-3, 0, 3]
REFINE_TOP_K = 5
candidates = [bpm * 0.5, bpm, bpm * 2.0]
aligned_bpm = min(
    candidates,
    key=lambda value: abs(np.log(value / reference_bpm))
)

pitch_shift, chroma_similarity = _compare_chroma(
```

```

        humming_profile["chroma"],
        segment_profile["chroma"],
    )

    style_tag, branch = _variant_label(
        effective_speed_factor,
        effective_pitch_shift,
        user_duration
    )

```

이 과정은 원곡 기반 배속 밈, 감속 편집, 피치 변형 버전 등을 판별하는 핵심 근거가 된다.

마지막 단계에서는 검색 결과를 사용자에게 이해하기 쉬운 형태로 제공한다.

YouTube 에서는 official, sped-up, shorts 등의 범주로 후보 영상을 검색하고, 제목·설명·길이·조회수·게시일 등의 메타데이터를 활용해 후보를 정렬한다. 이후 오디오 분석 결과와 YouTube 후보 정보를 바탕으로 GPT 가 최종 리포트를 생성한다. 또한 GPT 응답이 실패하거나 비어 있는 경우에는 기본 리포트를 출력하도록 설계하여 시스템의 안정성을 확보하였다.

따라서 본 시스템은 단순 곡 인식에 그치지 않고, 허밍과 가장 유사한 원곡 구간을 찾은 뒤 해당 음원이 숏폼 환경에서 어떤 방식으로 변형되어 사용되었는지까지 분석하여 제공하는 구조를 가진다.

## 핵심 코드

```

return {
    "official": _search("official music video", order="relevance"),
    "sped_up": _merge_results([...]),
    "shorts": _merge_results([...]),
}

response = _create_chat_completion(messages, max_tokens=GPT_MAX_TOKENS)
text = _extract_text(response)
if text:
    return text

if gpt_text and gpt_text.strip():
    st.markdown(gpt_text)

```

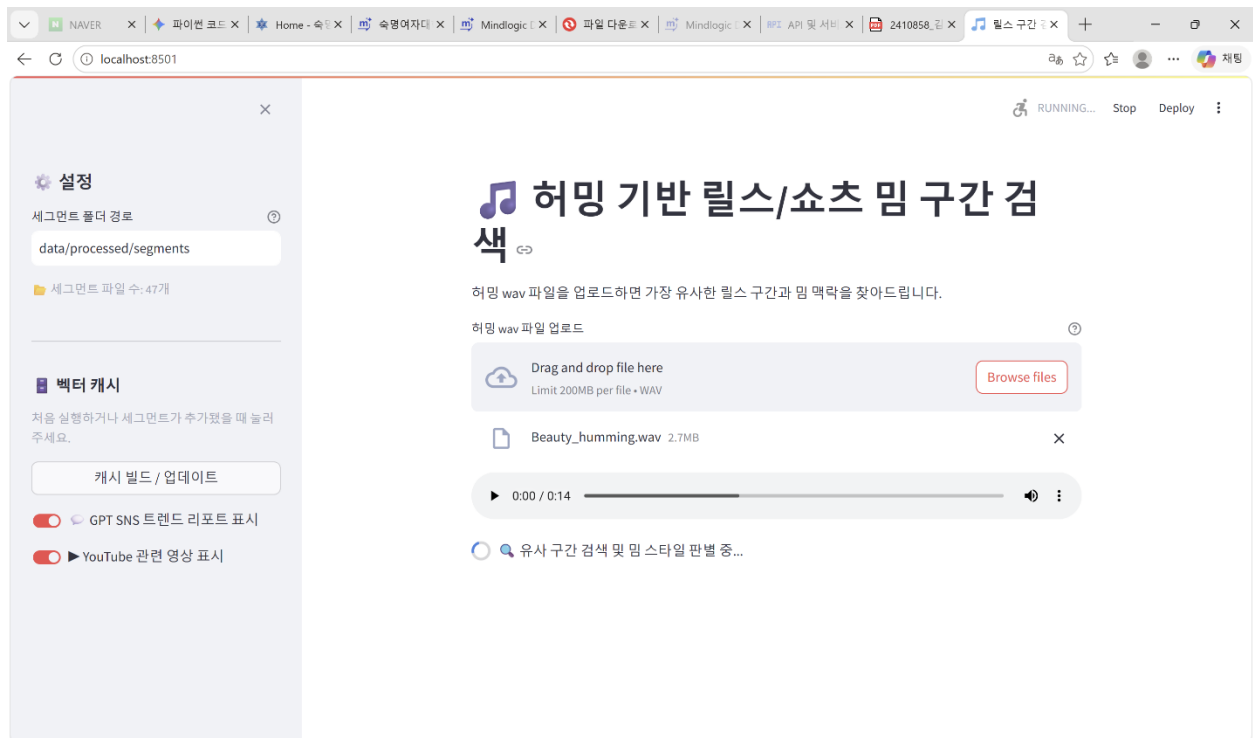
else:

```
st.warning("GPT 응답이 비어 있어 기본 리포트를 표시합니다.")
st.markdown(fallback_report)
```

이 구조 덕분에 “분석값 + 외부 후보 + 설명문”이 하나의 결과 화면으로 이어진다.

## 실행 결과와 해석

### 실행 결과 화면





NAVER

파이썬 코드

Home

속명어자다

Mindlogic

파일 다운로드

Mindlogic

API 및 서비스

2410858

원곡 구간

localhost:8501

채팅

RUNNING...

Stop

Deploy

1. 원곡 정보

Justin Bieber - Beauty And A Beat (feat. Nicki Minaj)

정밀 매칭  
88.5%

아티스트: Justin Bieber · 피쳐링: Nicki Minaj

원곡 파일: Beauty original · 총 길이 221.6초

2. 매칭된 변형 음원

원곡 기반 배속 밌 버전 (Speed Up Shorts)

원곡 구간보다 약 1.29배 빠르게 들립니다. chroma 기준 피치가 3반을 높아진 후보입니다. 멜로디/화성 유사도는 0.97입니다.

원곡 매칭 구간: 20초 ~ 30초

원곡에서 가장 가까운 구간

0:00 / 0:10

설정

세그먼트 폴더 경로  
data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

☒

 GPT SNS 트렌드 리포트 표시

☒

 YouTube 관련 영상 표시

NAVER

파이썬 코드

Home

속명어자다

Mindlogic

파일 다운로드

Mindlogic

API 및 서비스

2410858

원곡 구간

localhost:8501

채팅

RUNNING...

Stop

Deploy

3. 변형 분석

| Speed Factor | 입력 BPM   | 원곡 구간 BPM | Pitch Shift |
|--------------|----------|-----------|-------------|
| 1.29x        | 167      | 129       | +3 st       |
| Chroma 유사도   | 구간 길이 비율 | 원곡 커버리지   |             |
| 0.97         | 1.47     | 6.6%      |             |

분석 근거 자세히 보기

- 1차 원형 매칭 점수: 73.6%
- 변형 후보 정밀 매칭 점수: 88.5%
- 가장 가까운 변형 후보: speed 1.00x / pitch +3 st
- 허밍 길이: 14.7초
- 원곡 매칭 구간 길이: 10.0초
- 원곡 전체 길이: 221.6초
- Raw 허밍 BPM: 83
- 중앙 Pitch 비율: 1.54
- 입력 Key: F, 원곡 구간 Key: A

설정

세그먼트 폴더 경로  
data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

☒

 GPT SNS 트렌드 리포트 표시

☒

 YouTube 관련 영상 표시

설정

세그먼트 폴더 경로

data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

GPT SNS 트렌드 리포트 표시

YouTube 관련 영상 표시

## 4. 실제 사용 영상

### 허밍 분석과 가장 가까운 YouTube 후보

YouTube Data API 메타데이터 기반 후보입니다. 영상 오디오를 직접 비교한 결과는 아닙니다.

- [Beauty and a beat \(sped up\) - J.B ft. Nicky Minaj | Tutorial Dance Tiktok | #dance #trend #shorts](#) · Lento Lento  
후보 점수 20 · 조회수 255,384 · 2023-08-11T11:00:37Z · PT26S · 배속/리믹스 검색 후보, 입력 분석 speed +1.29x와 배속 키워드가 일치, 짧은 영상 길이, 조회수 기반 우선순위
- [Beauty and A Beat - Justin Bieber | #tiktok #dance #fyp #viral #song #lyrics #edit #ytshorts #trend](#) · sun\_moon\_K  
후보 점수 16 · 조회수 210,497 · 2026-05-25T13:10:24Z · PT15S · Shorts/Reels 검색 후보, 입력 분석 pitch +3st와 변형 키워드가 일치, 짧은 영상 길이, 조회수 기반 우선순위
- [beauty and a beat - justin bieber ft. nicki minaj \[edit audio\]](#) · YuNGFriz Edits  
후보 점수 15 · 조회수 1,773,592 · 2021-08-19T11:30:33Z · PT52S · 배속/리믹스 검색 후보, 입력 분석 pitch +3st와 변형 키워드가 일치, 짧은 영상 길이, 조회수 기반 우선순위

### 공식/MV

- [Justin Bieber - Beauty And A Beat \(Official Music Video\) ft. Nicki Minaj](#) · JustinBieberVEVO · 조회수 1,296,866,423 · 2012-10-12T15:55:06Z · PT4M53S
- [Justin Bieber ft. Nicki Minaj - Beauty And A Beat \(Lyrics-Letra\)](#) · Valdrew Lyrics · 조회수 88,335,417 · 2018-06-11T03:22:51Z · PT3M46S

설정

세그먼트 폴더 경로

data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

GPT SNS 트렌드 리포트 표시

YouTube 관련 영상 표시

### 공식/MV

- [Justin Bieber - Beauty And A Beat \(Official Music Video\) ft. Nicki Minaj](#) · JustinBieberVEVO · 조회수 1,296,866,423 · 2012-10-12T15:55:06Z · PT4M53S
- [Justin Bieber ft. Nicki Minaj - Beauty And A Beat \(Lyrics-Letra\)](#) · Valdrew Lyrics · 조회수 88,335,417 · 2018-06-11T03:22:51Z · PT3M46S
- [Justin Bieber - Beauty And A Beat ft. Nicki Minaj \(Official Audio\) ft. Nicki Minaj](#) · JustinBieberVEVO · 조회수 58,508,140 · 2012-06-18T15:40:07Z · PT3M49S
- [Justin Bieber, Nicki Minaj - Beauty And A Beat \(Lyrics\)](#) · Latin City · 조회수 31,929,403 · 2025-02-28T18:41:15Z · PT3M49S
- [Justin Bieber - Beauty And A Beat \(Lyrics Video\) ft. Nicki Minaj](#) · Deparea · 조회수 5,498,249 · 2026-04-18T05:33:08Z · PT4M3S

### 배속/리믹스

- [Beauty and a beat \(sped up\) - J.B ft. Nicky Minaj | Tutorial Dance Tiktok | #dance #trend #shorts](#) · Lento Lento · 조회수 255,384 · 2023-08-11T11:00:37Z · PT26S
- [Nicky Minaj & Justin Bieber - Beauty And A Beat](#) · BIGGA DRIP · 조회수 419,904 · 2023-08-28T08:50:39Z · PT21S
- [Justin Bieber - Beauty And A Beat ft. Nicki Minaj \(Sped up\)](#) · El gato · 조회수 685 · 2025-07-09T18:57:43Z · PT26S
- [beauty and a beat #princessamelia #dance](#) · Princess Amelia · 조회수 6,652,896 · 2024-02-

NAVER파이썬 코드Home - 속속영여자다Mindlogic파일 다운로드MindlogicAPI 및 서버2410858월스 구간+--oX

localhost:8501

RUNNING...StopDeploy

설정

세그먼트 폴더 경로data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

☒ GPT SNS 트렌드 리포트 표시

☒ YouTube 관련 영상 표시

08-19T11:30:33Z · PT12S

쇼츠/믹

- BEAUTY AND A BEAT - Justin Bieber ft Nicki Minaj #shorts · IQN ENTZ · 조회수 1,063 · 2026-06-01T12:00:23Z · PT12S
- Beauty and a Beat ~ Justin Bieber | #tiktok #dance #fyp #viral #song #lyrics #edit #ytshorts #trend · sun\_moon\_K · 조회수 210,549 · 2026-05-25T13:10:24Z · PT15S
- بلاقي beauty And a beat #مدرسة #songs #justinbieber #tiktok #elikuu · Emiko · 조회수 578,992 · 2026-05-25T15:45:29Z · PT14S
- Beauty and a beat Lyrics - Justin Bieber, Nicki Minaj | justinbieber #beautyandabeat #lyrics · Tusi Lyrics · 조회수 164,205 · 2026-05-27T07:59:07Z · PT34S
- Nicki Minaj and Justin Bieber flirt · Fame Blast · 조회수 102,349 · 2022-01-05T12:00:34Z · PT12S

5. 트렌드 리포트

☐ GPT 리포트 생성 중...

NAVER파이썬 코드Home - 속속영여자다Mindlogic파일 다운로드MindlogicAPI 및 서버2410858월스 구간+--oX

localhost:8501

Deploy

설정

세그먼트 폴더 경로data/processed/segments

세그먼트 파일 수: 47개

백터 캐시

처음 실행하거나 세그먼트가 추가됐을 때 눌러주세요.

캐시 빌드 / 업데이트

☒ GPT SNS 트렌드 리포트 표시

☒ YouTube 관련 영상 표시

5. 트렌드 리포트

1. 원곡과 매칭 근거

- 원곡: Justin Bieber - Beauty And A Beat (feat. Nicki Minaj) (원본 BPM 129, 길이 221.6s).
- 매칭 버전: "원곡 기반 배속 밈 버전 (Speed Up Shorts)" — 매칭 구간 20s~30s(Beauty\_original\_start00020s.wav).
- 허밍 근거: 허밍 길이 14.7s(커버리지 6.6%), 허밍-DB 유사도 73.6%, chroma 유사도 0.97 → 화성(음정 관계)은 원곡과 매우 유사.
- 속도/톤 비교: 허밍BPM-매칭 버전 BPM 167 vs 원곡 BPM 129 → BPM 비율 1.29 (추정 speed factor 1.29x). 추정 pitch shift +3 semitones, 중앙 pitch 비율 1.54.
- 길이 비교: 버전 길이 비율 1.47 → 매칭 버전이 원곡보다 길게 편집된 것으로 보임(반복/연장 가능성). 종합: 음정 구조는 보존(chroma 0.97)되었고, 명백한 배속(1.29x)과 +3반음 피치 이동이 적용된 편집판으로 판별됨.

2. 밈 맥락 분석

원곡 기반 배속(1.29x)은 즉시적인 에너지 상승과 템포감 강화를 만든다. 20~30초 구간을 14.7초 분량으로 잘라 쓰면 폭이 더 날카롭게 들리며 빠른 컷 전환, 드롭 직전 순간 강조, 코믹한 타이밍(예: 빠른 변신 샷) 등에 적합하다. +3반음 피치는 음색을 밝고 얇게 만들어 경쾌-경박한 뉘앙스를 주므로, 진지한 감정선보다는 유머/패러디-댄스 튜토리얼-속도감 강조 스타일에 유리하다. 단, 길이 비율이 1.47인 점은 배속과 함께 반복-연장이나 추가 리믹스가 섞였을 가능성을 시사하므로 편집 포인트를 확인해야 함.

3. YouTube 근거 링크

NAVER

파이썬 코드

Home - 속

속명여자다

Mindlogic

파일 다운로드

Mindlogic

API 및 서비

2410858.원

월스 구간

localhost:8501

채팅

Deploy

### 3. YouTube 근거 링크

- Justin Bieber - Beauty And A Beat (Official Music Video) ft. Nicki Minaj / JustinBieberVEVO / 조회수 1,296,867,546 / 길이 4:53  
[https://www.youtube.com/watch?v=Ys7-6\\_t70EQ](https://www.youtube.com/watch?v=Ys7-6_t70EQ)
  - 근거: 공식 원영상(원곡 기준 BPM-원음 비교용).
- Beauty and a beat (sped up) - J B ft. Nicky Minaj | Tutorial Dance Tiktok | #dance #trend #shorts / Lento Lento / 조회수 255,384 / 길이 26s  
<https://www.youtube.com/watch?v=h16c9INPI4E>
  - 근거: 배속 편집 사례(숏폼/댄스 튜토리얼 유형).
- BEAUTY AND A BEAT - Justin Bieber ft Nicki Minaj #shorts / IQN EdrIZ / 조회수 1,063 / 길이 12s  
<https://www.youtube.com/watch?v=1OXH6Etswho>
  - 근거: 숏폼/임 후보(짧은 클립 편집 예시).

### 4. 활용 가이드 (20s~30s 구간)

- 댄스 튜토리얼 샷
  - 영상 유형: 1인 숏댄스(파트별 동작 분해)
  - 컷 편집: 29컷에서 각 동작을 0.51초로 빠르게 매치(템포에 동기화)
  - 연출 포인트: 자막으로 "속도 업 꿀팁" 표시, 비트에 맞춰 손동작 강조 이펙트 삽입.
- 변신/전환 코미디

NAVER

파이썬 코드

Home - 속

속명여자다

Mindlogic

파일 다운로드

Mindlogic

API 및 서비

2410858.원

월스 구간

localhost:8501

채팅

Deploy

- 댄스 튜토리얼 샷
  - 영상 유형: 1인 숏댄스(파트별 동작 분해)
  - 컷 편집: 29컷에서 각 동작을 0.51초로 빠르게 매치(템포에 동기화)
  - 연출 포인트: 자막으로 "속도 업 꿀팁" 표시, 비트에 맞춰 손동작 강조 이펙트 삽입.
- 변신/전환 코미디
  - 영상 유형: 전신 → 코스트룸 변신(점프/스냅으로 전환)
  - 컷 편집: 컷 점프(히트 타임에 컷 전환), 모션 블러로 연결감 유지
  - 연출 포인트: +3반음으로 밝아진 톤을 이용해 표정 과장, 자막으로 타이밍(예: "역대급 변신!") 삽입.
- 텍스트 모음/하이라이트 리액션
  - 영상 유형: 제품/임 클립 하이라이트(빠르게 넘기는 형식)
  - 컷 편집: 0.5-1s 썸스냅 컷, 하단에 빠른 라인 자막(한 문장씩)
  - 연출 포인트: 비트마다 텍스트 애니메이션 출현, 루프 가능하도록 14s 이내 구성(커버리지 6.6% 활용).

(주의) 위 분석은 제공된 허밍-메타데이터-유튜브 검색 결과 기준의 실시간 유행 후보 데이터에 한함.

전체 세그먼트 유사도 보기

rude\_original\_start00080s.wav — 81.2%

Beauty\_original\_start00100s.wav — 77.1%

Beauty\_original\_start00010s.wav — 74.0%

사용자가 제공한 실행 화면 기준으로, 입력된 Beauty\_humming.wav 는 Justin Bieber - Beauty And A Beat (feat. Nicki Minaj)에 매칭되었고, 최종 정밀 매칭 점수는 88.5%, 신뢰도는 “보통”으로 표시되었다. 매칭된 변형 음원 태그는 원곡 기반 배속 밈 버전 (Speed Up Shorts)였으며, 원곡에서 가장 가까운 구간은 20 초~30 초였다. 결과 화면은 이 입력을 단순 원곡 인식이 아니라 “원곡 기반 스포츠 배속·피치 변형”으로 판정했다.

실행 화면에서 확인된 주요 수치는 아래와 같다.

| 지표                 | 실행 결과 | 해석                               |
|--------------------|-------|----------------------------------|
| 정밀 매칭 점수           | 88.5% | 변형 보정 후 원곡 연관성이 높음               |
| 1 차 원형 매칭 점수       | 73.6% | 원형 상태만으로는 충분히 높지 않음              |
| Speed Factor       | 1.29x | 원곡 구간 대비 명확한 배속                  |
| 입력 BPM             | 167   | 입력 허밍의 체감 템포가 높음                 |
| 원곡 구간 BPM          | 129   | 원곡 기준 템포                         |
| Pitch Shift        | +3 st | 스포츠형 밝고 가벼운 피치 상승                |
| Chroma 유사도         | 0.97  | 멜로디/화성 구조 보존 수준이 매우 높음           |
| 구간 길이 비율           | 1.47  | 10 초 원곡 구간이 입력 14.7 초로 확장·반복된 형태 |
| 원곡 커버리지            | 6.6%  | 전체 곡 중 짧은 하이라이트 구간 사용            |
| 중앙 Pitch 비율        | 1.54  | 실제 음높이도 상승 경향                    |
| 입력 Key / 원곡 구간 Key | F / A | 조성 차이가 관측됨                       |

이 수치들은 화면에 표시된 변형 분석과 상세 근거를 요약한 것이다. 지표의 정의와 계산 방식 자체는 검색·분석 코드에 구현되어 있다

이 결과를 해석하면, 해당 입력은 원곡 구조를 거의 유지하면서도 속도와 피치를 같이 끌어올린 스포츠형 편집에 가깝다. 특히 chroma similarity 0.97 은 멜로디/화성의 뼈대가 살아 있음을 의미하고, speed factor 1.29x 와 pitch shift +3st 는 스포츠에서 에너지와 밝기를 끌어올리는 전형적인 편집 방향과 맞물린다. 여기에 구간 길이 비율 1.47 까지 고려하면, 단순한 정속 재생이 아니라 반복·연장·컷 편집이 결합된 짧은 릴스용 오디오일 가능성이 높다. 이 해석은 결과 화면의 설명 문구와 GPT 리포트에도 일관되게 반영되었다.

특히 주목할 부분은 “원형 점수는 낮지만 변형 점수는 높았다”는 점이다. 이는 이 프로젝트가 단순 최고 코사인 유사도 검색기가 아니라는 뜻이다. 전체 세그먼트 유사도 화면에서는 다른 곡인 RUDE!의 세그먼트도 높은 1 차 후보로 보였지만, 최종 선택은 speed/pitch 보정을 거친 Beauty And A Beat 의 20~30 초 구간으로 수렴했다. 즉, 본 시스템은 “무슨 곡이 비슷한가”보다 “어떤 원곡이 변형된 상태로 더 설명력이 높은가”를 찾는다. 이 부분이 실험 결과에서 가장 설득력 있게 드러난 성과다.

YouTube 연동 결과도 프로젝트 목적에 잘 맞았다. 결과 화면에는 “허밍 분석과 가장 가까운 YouTube 후보” 상위 3 개가 제시되었고, 대표 후보는 Beauty and a beat (sped up) - J B ft. Nicky Minaj | Tutorial Dance Tiktok 처럼 배속·쇼츠 키워드를 모두 포함하는 영상이었다. 별도의 범주 목록에는 공식 MV, 배속/리믹스, 쇼츠/밈 영상이 분리되어 표기되어, 원곡 근거와 숏폼 활용 근거를 동시에 제시했다. 다만 화면에도 명시되어 있듯이 랭킹은 영상 오디오 직접 비교가 아니라 Data API 메타데이터 기반 후보라는 점은 분명히 유지되었다.

트렌드 리포트는 실제로 성공적으로 생성되었다. 화면에는 원곡과 매칭 근거, 밈 맥락 분석, YouTube 근거 링크, 활용 가이드의 네 부분이 순서대로 출력되었고, 활용 가이드에서는 댄스 튜토리얼 샷, 변신/전환 코미디, 텍스트 모음·하이라이트 리액션 같은 구체적 활용 시나리오가 제안되었다. 이는 gpt.py 프롬프트가 요구하는 출력 구조와 정확히 맞아떨어지는 결과다.

## 핵심 코드

```
variation_analysis = {
    "transform_match_score": transform_score,
    "base_match_score": best_score,
    "speed_factor": effective_speed_factor,
    "duration_ratio": duration_ratio,
    "pitch_shift_semitones": effective_pitch_shift,
    "chroma_similarity": chroma_similarity,
    "coverage_ratio": user_duration / db_duration if db_duration > 0 else 0.0
},

st.metric("Speed Factor", f"{speed_factor:.2f}x" if speed_factor else "N/A")
st.metric("입력 BPM", f"{ms['bpm_humming']:.0f}" if ms.get("bpm_humming") else "N/A")
st.metric("원곡 구간 BPM", f"{ms['bpm_original_segment']:.0f}" if ms.get("bpm_
```

```
original_segment") else "N/A")
st.metric("Pitch Shift", f"{pitch_shift:+d} st" if pitch_shift is not None else "N/A")
```

이 코드가 결과 화면의 핵심 숫자들을 실제로 만들어 내고, 사용자에게 읽기 쉬운 형태로 보여 주는 부분이다.

## 한계와 개선 방향

현재 시스템은 허밍 기반 곡 탐색과 릴스 후보 추천을 수행하지만 몇 가지 한계가 있다. 가장 큰 한계는 YouTube 영상의 실제 오디오를 비교하지 않는다는 점이다. 현재는 제목, 설명, 길이, 조회수 등의 메타데이터를 기반으로 후보를 선정하므로, 결과는 실제 사용 영상이라기보다 가능성이 높은 후보에 가깝다. 향후에는 후보 영상의 오디오를 추출하여 현재의 유사도 분석 파이프라인을 다시 적용하면 정확도를 높일 수 있다. 또한 짧거나 잡음이 많은 허밍에서는 BPM 과 pitch 추정이 불안정할 수 있다. 현재는 librosa 의 beat\_track 과 piptrack 을 사용하므로 입력 품질에 영향을 받는다. 이를 개선하기 위해 잡음 제거, 허밍 전용 전처리, 보다 정교한 key·pitch 추정 기법을 적용할 수 있다. 곡 메타데이터를 수동으로 등록해야 한다는 점도 확장성의 한계이다. 현재는 카탈로그 파일을 직접 수정해야 하므로, 향후 외부 음악 메타데이터 API 연동이나 관리 UI 를 통해 등록 과정을 자동화할 필요가 있다. 구현 측면에서는 특징 벡터 차원 설명과 실제 계산 방식이 일부 다르며, 세그먼트 분할 방식에 대한 설명도 코드와 완전히 일치하지 않는다. 따라서 문서와 코드 설명을 통일해 재현성과 유지보수성을 높일 필요가 있다. 마지막으로 사용자 인터페이스에서는 1 차 유사도 점수만 표시되어 최종 결과와 차이가 발생할 수 있다. 이를 해결하기 위해 base score 와 refined score 를 함께 제공하면 결과의 해석이 더 명확해질 것이다.

아래는 한계와 개선안을 정리한 표다.

| 한계                       | 현재 구현 근거                   | 개선 방향                       |
|--------------------------|----------------------------|-----------------------------|
| YouTube 영상 오디오 직접 비교 없음  | 메타데이터 기반 랭킹                | 후보 영상 오디오 추출 후 동일 파이프라인 재적용 |
| 짧은 허밍에서 BPM/pitch 흔들림 가능 | beat_track, piptrack 기반 추정 | 허밍 전용 전처리, key/pitch 안정화    |

| 한계                  | 현재 구현 근거         | 개선 방향                       |
|---------------------|------------------|-----------------------------|
| 곡 메타데이터 수동 등록       | 수동 카탈로그 파일       | 외부 메타데이터 API 연동 또는<br>관리 UI |
| 결과 설명과 구현 일부<br>불일치 | 45 차원/비중첩 분할     | 문서·주석·출력 설명 통합              |
| 전체 유사도 화면의 해석<br>혼동 | 1 차 점수만 노출       | base/refined score 동시 표기    |
| 검색 속도 확장성           | 모든 세그먼트 순회<br>비교 | 벡터 인덱스 도입                   |

표의 모든 항목은 현재 코드 구조와 실행 화면에서 바로 확인되는 내용에 기반한다.

## 핵심 코드

```
if gpt_text and gpt_text.strip():
    st.markdown(gpt_text)
else:
    st.warning("GPT 응답이 비어 있어 기본 리포트를 표시합니다.")
    st.markdown(fallback_report)

for seg, score in sorted(result["all_scores"].items(), key=lambda x: -x[1]):
    icon = "■" if seg == result["best_segment"] else "□"
    st.write(f"{icon} `{seg}` - **{score*100:.1f}%**")
```

첫 번째 코드는 실패 복구 구조를, 두 번째 코드는 현재 전체 유사도 화면이 “1 차 점수 중심”이라는 점을 보여 준다. 개선 포인트가 코드에서 바로 드러난다는 뜻이다.

## 결론

이 프로젝트는 “허밍만으로 스포츠 변형 음원을 설명한다”는 목표를 비교적 설득력 있게 달성했다. 실제 실험에서 시스템은 Beauty And A Beat 릴스형 허밍 입력을 Justin Bieber - Beauty And A Beat 의 20 초~30 초 구간과 연결했고, 1 차 원형 매칭 73.6% → 변형 정밀 매칭 88.5%로 점수를 끌어올리며, 1.29x 배속과 +3st 피치 상승을 해석 가능한 지표로 제시했다. 이는 원곡만 저장해도 스포츠형 변형 음원을 상당 부분 설명할 수 있음을 보여 준다.



무엇보다 이 프로젝트의 강점은 결과를 “왜 그렇게 판단했는지”까지 함께 보여 준다는 데 있다. 특징 벡터, BPM 보정, chroma 기반 pitch shift, top-K 변형 재매칭, YouTube 후보, GPT 리포트가 하나의 UI 로 이어지면서, 단순 인식기가 아니라 설명 가능한 음악 분석 도구에 가까운 형태를 갖추었다. 앞으로 YouTube 오디오 직접 비교, 벡터 인덱스, 자동 메타데이터 등록, UI 신뢰도 표기를 더하면 완성도 높은 스포츠 음원 분석 시스템으로 확장될 가능성이 크다.

---